NewsShield

Code Documentation

AI Content Detection · v1.0

Application: ai_detection_streamlit.py

---

1. Overview

NewsShield is a Streamlit web application providing two machine-learning services for news article analysis. It is available both as a locally-runnable script and as a hosted dashboard:

```
https://newsshield.streamlit.app
```

To run the application locally, execute the following in the CLI from the project root directory:

```
streamlit run ai_detection_streamlit.py
```

The application is structured as a single-page interface with a styled top navigation bar, a hero section, and two independent analysis panels rendered sequentially on the same page.

Note: this project is still in progress and the documentation is not yet complete.

---

2. Project Objectives

The project has two primary machine-learning objectives:

Objective 1 — Detecting AI-Generated News

- Build a binary classifier to distinguish human-written news articles from AI-generated content.
- Training data was collected via a Scrapy web scraper targeting Wikipedia's Current Events portal (https://en.wikipedia.org/wiki/Portal:Current_events/) covering years 2004 to 2025 inclusive.
- Processed data is stored in the Data/ folder.
- The trained artefacts (vectoriser and model) are stored in the arvix_data/ folder.

Objective 2 — Multi-Label Topic Classification

- Build a multi-label classifier to assign news articles to one or more topic categories.
- Data sourced from Roy et al. (2025), A Comprehensive Dataset for Human vs. AI Generated Text Detection. Available at: https://arxiv.org/html/2510.22874v1.
- This dataset is stored within the arvix_data/ folder.

Reference

Roy, R., Imanpour, N., Aziz, A., et al. (2025). A Comprehensive Dataset for Human vs. AI Generated Text Detection. Arxiv.org. Available at: https://arxiv.org/html/2510.22874v1.

---

3. Dependencies & Imports

The full list of pinned dependencies for ai_detection_streamlit.py is specified in requirements.txt. Install all dependencies with:

```
pip install -r requirements.txt
```

3.1 Direct Imports in ai_detection_streamlit.py

The following packages are imported directly by the main application script:

Package & Version	Role in app
streamlit 1.53.1	Core framework for the web UI and interactive widgets.
streamlit-shap 1.0.2	Renders SHAP explanation plots (waterfall, bar) inside Streamlit via st_shap().
ai_detection (local)	Custom module — defines the ai_detector class.
classify_news (local)	Custom module — defines the classifier class.

3.2 Full requirements.txt

The following packages are required across the full project (including both ML modules):

Package & Version	Purpose
imbalanced-learn 0.14.1	Oversampling / undersampling utilities for handling class imbalance during model training.
markdown 3.10	Markdown parsing — used internally by Streamlit and its dependencies.
markdown-it-py 4.0.0	Markdown renderer; Streamlit dependency.
markupsafe 3.0.3	Safe HTML string handling; Jinja2 / Streamlit dependency.
matplotlib 3.10.6	Plotting library; used by SHAP for rendering explanation graphics.
nltk 3.9.1	Natural Language Toolkit — text preprocessing utilities.

numpy 1.26.4	Numerical array operations; underpins scikit-learn, SHAP, and pandas.
pandas 2.3.3	DataFrame construction and manipulation for prediction outputs.
pip 26.0.1	Package installer — pinned version for reproducibility.
plotly 6.3.0	Interactive charting library; available for future visualisation extensions.
pluggy 1.6.0	Plugin system used internally by pytest and other tooling.
scikit-learn 1.7.2	Core ML library providing TF-IDF vectorisation, classifiers, and the MultiLabelBinarizer.
scikit-multilearn 0.2.0	Multi-label classification wrappers; used for the topic classifier.
scikit-optimize 0.10.2	Bayesian hyperparameter optimisation; used during model training.
shap 0.48.0	SHAP (SHapley Additive exPlanations) — computes and renders model explanations.
streamlit 1.53.1	Web application framework for the dashboard UI.
streamlit-shap 1.0.2	Streamlit component for rendering SHAP plots inline.
xgboost 3.0.5	Gradient boosted tree library; used as the underlying classifier for one or both ML models.

Note: both ai_detector and classifier are local modules (ai_detection.py and classify_news.py) and are not installed via pip — they must be present in the project working directory.

---

4. Page Configuration

```
st.set_page_config(page_title='NewsShield', page_icon='🛡', layout='wide',
initial_sidebar_state='collapsed')
```

Parameter	Value & Purpose
page_title	Browser tab title: 'NewsShield'.
page_icon	Favicon emoji shown in the browser tab.
layout	'wide' — uses full browser width rather than the Streamlit default centred column.

initial_sidebar_state	'collapsed' — sidebar is hidden on load; the app does not use it.

---

5. Custom CSS (Styling Layer)

A large inline <style> block is injected via st.markdown(..., unsafe_allow_html=True). It imports three Google Fonts and defines all visual components. Key design tokens:

Token	Value & Role
Background	#0f1117 — near-black base canvas.
Surface	#161820 — slightly lighter panels, nav bar, inputs.
Border	#2a2d3a — subtle dark border for cards and inputs.
Body text	#e2e2e2 — off-white primary text.
Muted text	#7a7f99 — secondary / descriptive text.
Accent (button)	#3d4270 — indigo-blue interactive colour.
Font: display	DM Serif Display — headings and hero text.
Font: body	DM Sans (weights 300/400/500) — all body copy.
Font: mono	DM Mono (weights 400/500) — labels, tags, code.

5.1 Styled Component Inventory

The CSS targets the following UI regions:

- .topbar / .topbar-brand / .topbar-tag — fixed top navigation bar with logo and version tag.
- .hero — large serif headline and descriptive paragraph.
- .label — small-caps monospaced section labels.
- .thin-rule — 1 px horizontal divider.
- .about-body — descriptive text with stronger weight for key terms.
- [data-testid='stTextArea'] — overrides Streamlit's default textarea to match the dark theme.
- [data-testid='stButton'] button[kind='primary'] — custom primary button style with hover state.
- .verdict-wrap (.ai-card / .human-card) — coloured result card (red tones for AI, green tones for human).
- .conf-bar-track / .conf-bar-fill-* — thin confidence progress bar inside the verdict card.

- .topic-tag / .topic-tag-row — pill-style topic label chips.
- .topic-bar-row / .topic-bar-label / .topic-bar-track / .topic-bar-fill / .topic-bar-pct — horizontal confidence bars for topic scores.
- .shap-info — info box displayed below the SHAP waterfall chart.

---

6. Page Layout Structure

The page is composed of the following sections rendered in order:

Section	Content & Notes
Top Navigation Bar	HTML div .topbar — brand name left, version tag right.
Hero Section	HTML div .hero — main headline and sub-copy.
Thin Rule	Visual separator (<hr> equivalent).
About Section	Two-column layout: st.image() on the left (ny_times.jpg), descriptive text on the right. Columns use ratio [1, 2] with large gap.
Thin Rule	Visual separator.
AI Detection Panel	Text input, Analyse button, verdict card, SHAP chart (conditional on button press).
Thin Rule	Visual separator.
Topic Classification Panel	Text input, Classify button, topic tags, confidence bars (conditional on button press).

---

7. Section 1 — AI Detection

7.1 Input

A multiline text area (key='ai_input') accepts free-form article text. The placeholder reads 'Paste a news article or excerpt here…'. Height is fixed at 220 px.

7.2 Trigger

A primary-styled button labelled 'Analyse Text' (key='run_ai') triggers analysis. Nothing is rendered until the button is pressed and the text area is non-empty. If the button is pressed with an empty field, a Streamlit st.warning() banner is shown.

7.3 Model Invocation

When triggered, the application:

- Instantiates ai_detector(ai_text) — passes the raw article string.
- Calls detector.predict() → returns a string label (e.g. 'AI-generated' or 'Human-written').
- Calls detector.predict_proba() → returns a confidence value (float 0–1 or percentage string).

7.4 Confidence Parsing

The confidence value is normalised defensively:

- The raw value is cast to float and stripped of any '%' character.
- If the value is ≤ 1.0, it is multiplied by 100 (converting from proportion to percentage).
- Display string (e.g. '87.3%') and bar-fill width string (e.g. '87.3%') are computed.
- On any parsing error, the raw string is used as-is and the bar width defaults to '0%'.

7.5 Verdict Card

A styled HTML card is rendered with the following logic:

Variable	Logic
is_ai	True if 'human' is NOT in the prediction string (case-insensitive).
card_class	'ai-card' (red) or 'human-card' (green) CSS class.
bar_class	'conf-bar-fill-ai' (red bar) or 'conf-bar-fill-human' (green bar).
icon	🤖 for AI, ✍️ for Human.
title	'AI-Generated' or 'Human-Written'.
sub	Confidence · <parsed_percentage>.

7.6 Explanation (SHAP)

After the verdict card, an explanation section is rendered:

- detector.explain() is called — returns a SHAP plot object.
- st_shap(explain_plot) renders the interactive waterfall plot from the streamlit_shap library.
- A .shap-info box below explains the colour encoding: red bars = push toward human-written; blue bars = push toward AI-generated.

---

8. Section 2 — Topic Classification

8.1 Input

A separate multiline text area (key='topic_input') with identical dimensions and placeholder to the AI Detection panel. The two panels are fully independent — they do not share state.

8.2 Trigger

A primary-styled button labelled 'Classify Topics' (key='run_topic') triggers classification. An st.warning() is shown if the field is empty on press.

8.3 Model Invocation

When triggered:

- classifier(text=topic_text) is instantiated.
- clf.predict() → returns labels_df, a DataFrame where column names are the predicted topic labels.
- clf.predict_proba() → returns proba_df, a DataFrame where column names are all candidate topic labels and values are confidence scores (floats 0–1).

8.4 Predicted Topic Tags

The column names of labels_df are used as the set of predicted topics. Each topic is rendered as a styled chip (.topic-tag) inside a flex row (.topic-tag-row). If no columns are returned, a 'No topics matched' message is displayed.

8.5 Confidence Score Bars

If proba_df is non-empty, a horizontal bar chart is rendered in HTML for each topic in proba_df.columns:

- topic-bar-label — topic name, truncated with ellipsis at 220 px.
- topic-bar-track — grey background track.
- topic-bar-fill — blue fill, width = min(score * 100, 100)%.
- topic-bar-pct — percentage label, right-aligned.

---

9. State & Session Management

The application uses Streamlit's default re-run model — the entire script is re-executed on every user interaction. No explicit st.session_state is used. Results are only displayed within the conditional blocks (if run_ai and ai_text.strip(): ...) immediately following each button, so they persist visually only as long as no other interaction triggers a re-run that would clear them.

Practical implication: pressing 'Classify Topics' will re-run the script and clear any previously rendered AI Detection results if the user has not yet interacted with that panel in the current execution.

---

10. Error Handling

Scenario	Handling
Empty text input	Guarded by .strip() check; st.warning() shown, model not called.
Confidence parsing failure	try/except around float conversion; falls back to raw string and 0% bar width.
Empty proba_df	if not proba_df.empty: guard prevents rendering the confidence section.
Model errors	No explicit try/except wraps detector or classifier calls — exceptions will surface as Streamlit error banners.

---

11. Key Variables Reference

Variable	Type & Description
ai_text	str — raw text pasted into the AI Detection text area.
run_ai	bool — True when 'Analyse Text' button is pressed.
detector	ai_detector instance — wraps the AI detection model.
prediction	str — human-readable label returned by detector.predict().
confidence	float | str — raw confidence from detector.predict_proba().
conf_num	float — normalised confidence as a percentage (0–100).
conf_display	str — formatted display string, e.g. '87.3%'.
bar_pct	str — CSS width string for the confidence bar fill.
is_ai	bool — True if prediction does not contain 'human'.
explain_plot	SHAP plot object — returned by detector.explain().
topic_text	str — raw text pasted into the Topic Classification text area.
run_topic	bool — True when 'Classify Topics' button is pressed.

clf	classifier instance — wraps the topic classification model.
labels_df	pd.DataFrame — columns are predicted topic labels.
proba_df	pd.DataFrame — columns are all topics; values are confidence scores.
topics_found	list[str] — list of predicted topic names from labels_df.columns.

---

12. Module: ai_detection.py

This module defines the ai_detector class, which encapsulates all logic for preprocessing input text, running inference, and generating SHAP explanations. Two objects are loaded at module level (outside the class) so they are only deserialised once per process:

```
vectoriser = joblib.load('arvix_data/ai_vectoriser.pkl')
model      = joblib.load('arvix_data/ai_detector.pkl')
```

Both are then assigned to instance attributes inside __init__, making them accessible via self throughout the class.

12.1 Constructor — __init__(self, text: str)

Accepts a raw article string and prepares it for inference.

Step	Detail
Input cleaning	re.sub('[^a-zA-Z0-9]+', ' ', text).strip() — strips all non-alphanumeric characters (punctuation, special symbols) and collapses runs of whitespace into single spaces.
self.text	Stores the cleaned string.
self.vectoriser	Reference to the module-level TF-IDF vectoriser.
self.model	Reference to the module-level classifier.
self.text_list	Wraps self.text in a list: [self.text] — required by sklearn's transform API.
self.text_tfidf	Sparse TF-IDF matrix produced by self.vectoriser.transform(self.text_list). Shape: (1, n_features).

12.2 predict(self) — str

Runs the classifier and returns a human-readable verdict string.

- Calls self.model.predict(self.text_tfidf) — returns an array with a single integer label.

- Label 0 — returns 'This is likely a human generated text'.
- Any other label — returns 'This is likely an AI generated text'.

Note: the ai_detection_streamlit.py caller checks whether the returned string contains the substring 'human' (case-insensitive) to determine the is_ai flag and select card styling.

12.3 predict_proba(self) — float

Returns the confidence score for the winning class.

- Calls self.model.predict_proba(self.text_tfidf) — returns a (1, 2) probability array.
- Constructs a DataFrame with columns ['human_generated', 'AI_generated'].
- Filters to columns where all values are >= 0.5 — isolating the dominant class.
- Returns the scalar float at [0][0] of the filtered DataFrame — the probability of the predicted class.

Important: if the model outputs exactly 50/50, both columns are filtered out and .values[0][0] will raise an IndexError. This edge case is not currently guarded.

12.4 explain(self) — shap plot object

Generates a SHAP waterfall explanation for the input.

- Instantiates shap.Explainer with the model, the dense input array (self.text_tfidf.toarray()), and feature names from the vectoriser vocabulary.
- Calls explainer(self.text_tfidf.toarray()) to compute SHAP values — output shape: (1, n_features, n_classes).
- Slices shap_values[:,:,0][0] to extract the explanation for class index 0 (human_generated) for the single input sample.
- Returns a shap.plots.waterfall plot object, rendered in the app via st_shap().

Note: class index 0 corresponds to human_generated. The app's info box labels red bars as 'push toward human-written' and blue bars as 'push toward AI-generated', consistent with this class ordering.

12.5 Persisted Artefacts

File	Description
arvix_data/ai_vectoriser.pkl	Serialised sklearn TF-IDF vectoriser. Must be fitted on the same vocabulary used during training.
arvix_data/ai_detector.pkl	Serialised sklearn-compatible classifier. Must expose .predict(), .predict_proba(), and be compatible with shap.Explainer.

Both files are loaded at module import time. If either file is missing or corrupt, the import will fail with a joblib exception before any Streamlit UI is rendered.

---

13. Module: classify_news.py

This module defines the classifier class, which handles multi-label topic classification of news articles. Three objects are loaded at module level to avoid repeated deserialisation on each instantiation:

```
vectoriser    = joblib.load('data/tfidf_vectorizer.pkl')
topics        = joblib.load('data/mlb.pkl')
topic_hashmap = topics.classes_
model         = joblib.load('data/news_topic_classifier.pkl')
```

topic_hashmap is a NumPy array of topic label strings derived from the fitted MultiLabelBinarizer (mlb.pkl). It is used as the column names for all prediction DataFrames throughout the class.

13.1 Constructor — __init__(self, text: str)

Accepts a raw article string and prepares the TF-IDF representation.

Step	Detail
Input cleaning	re.sub('[^a-zA-Z0-9]+', ' ', text).strip() — identical preprocessing to ai_detector: removes all non-alphanumeric characters and normalises whitespace.
self.text	Stores the cleaned string.
self.vectoriser	Reference to the module-level TF-IDF vectoriser.
self.text_tfidf	Sparse TF-IDF matrix from self.vectoriser.transform([self.text]). Shape: (1, n_features). A debug print statement outputs this value to stdout on every instantiation.
self.model	Reference to the module-level multi-label classifier.

Note: the constructor contains a print(f'text tfidf: {self.text_tfidf}') debug statement. This outputs to the terminal on every instantiation and should be removed before production deployment.

13.2 predict(self) — pd.DataFrame

Returns a DataFrame containing only the predicted (positive) topic labels.

- Calls self.model.predict(self.text_tfidf) — returns a binary (1/0) matrix of shape (1, n_topics).
- Constructs a DataFrame with topic_hashmap as column names.
- Filters to columns where all values equal 1 — retaining only topics the model predicts as present.
- Returns the filtered DataFrame. Column names become topics_found = list(labels_df.columns) in the app.

If no topics are predicted, the returned DataFrame has no columns and topics_found will be an empty list, triggering the 'No topics matched' message in the UI.

13.3 predict_proba(self) — pd.DataFrame

Returns a DataFrame of confidence scores for all topics above the 0.5 threshold.

- Calls self.model.predict_proba(self.text_tfidf) — returns a probability matrix of shape (1, n_topics).
- Constructs a DataFrame with topic_hashmap as column names.
- Filters to columns where all values are >= 0.5, matching the threshold used in predict().
- Returns the filtered DataFrame. The app reads proba_df[topic].iloc[0] per column to render each confidence bar.

Note: predict() and predict_proba() apply the 0.5 threshold independently. A topic could theoretically appear in one result but not the other if a score sits precisely at the boundary.

13.4 explain(self) — not implemented

The method body is entirely commented out and returns None via pass. The commented code outlines the intended implementation:

- A shap.Explainer would wrap self.model.predict_proba with feature names from the vectoriser vocabulary.
- SHAP values would be computed on the dense input array.
- A shap.plots.bar chart would be generated, sliced to class index 0.

This method is not called anywhere in ai_detection_streamlit.py and can be safely ignored at runtime. The commented block serves as a template for future development.

13.5 Persisted Artefacts

File	Description
data/tfidf_vectorizer.pkl	Serialised sklearn TF-IDF vectoriser. Must share the same vocabulary as the one used during model training.
data/mlb.pkl	Serialised sklearn MultiLabelBinarizer. Its .classes_ attribute provides the ordered array of topic label strings used as column names.
data/news_topic_classifier.pkl	Serialised multi-label classifier. Must expose .predict() and .predict_proba(), both returning arrays of shape (1, n_topics).

All three files are loaded at module import time from the data/ directory relative to the working directory. Missing or corrupt files will raise a joblib exception before the Streamlit UI initialises.

---

14. External Assets

Asset	Notes

ny_times.jpg	Static image displayed in the About section. Must be present in the working directory alongside ai_detection_streamlit.py.
Google Fonts CDN	DM Serif Display, DM Sans, DM Mono — loaded at runtime via @import in the injected CSS. Requires an internet connection on first load.

---

15. Notes for Developers

- Adding new models: create a new class in a separate module following the same .predict() / .predict_proba() interface, instantiate it in a new conditional block, and replicate the verdict rendering pattern.
- Session state: if results need to persist across re-runs (e.g. after a second button press), wrap key outputs in st.session_state.
- SHAP compatibility: st_shap expects a SHAP Explanation or plot object. Ensure detector.explain() returns a compatible type for the installed version of streamlit-shap.
- Model loading performance: for production use, wrap model loading in @st.cache_resource to avoid reloading on every re-run.
- Confidence direction note: the shap-info box states red = human, blue = AI. Ensure the underlying SHAP model's class ordering matches this labelling, or update the info text accordingly.